From memory to recursive self-improvement

Agents that learn from outcomes, agents that only remember, and a model trained on what they learned, when the tasks never repeat and the rules change

SenseLab Research

https://sense-lab.ai

Preprint, September 2026

Abstract

Teams that put LLM agents into production add a memory layer early, so that the agent does not re-derive on Tuesday what it worked out on Monday. Memory solves recall. It does not tell the agent whether what it recalled was right, and in production that question comes up constantly: policies change, vendors change their APIs, a fix that worked last quarter is wrong this quarter. An agent that cannot adapt does not fail at random when this happens. It applies the old answer to every instance of the affected task, with full confidence, until a human notices. We measured how much that matters on a benchmark built to resemble production: tasks that are new every time, domains with hidden quirks where the textbook answer is wrong, a rule change at episode 20 that turns a learned fix into a wrong one, fleets of agents on one shared store, and newcomers joining mid-stream. Six scenarios, two frontier models, single agents and fleets of six, about 35,000 agent episodes. We compare no memory, a hosted memory product (Mem0, with its outcome-feedback API wired), and a layer that learns from outcomes (SenseLab), crediting every outcome back to the memories and actions that produced it. Before the rule change, memory and outcome learning both raise first-attempt success from 0.53 to above 0.8 over twenty episodes; no memory stays flat. After the change, memory stops helping. On the affected task classes Mem0 and no memory both sit at 0.38 first-attempt success for the remaining forty episodes, and the memory arms apply the stale fix on twice as many first attempts as the agent with no memory at all; the agent whose layer learns from outcomes goes from 0.36 to 0.69 by the last block, drops its stale-pick rate by half, and escalates a quarter to a third fewer tasks to humans. The same platform with its outcome loop switched off loses most of that recovery, which is how we know the loop is what does the work. A fleet of six sharing one store inherits the recovery. So does a newcomer that joins after the change, and followers joining an agent that learned alone solve 0.84 of their first exposures against 0.67 on the memory product and 0.46 with no memory. The outcome record this loop builds is also a training set. A small model trained on the frontier agent's sealed traces and served with the same memory solves 0.94 and 0.92 of tasks on two of these scenarios, under the rule change its teacher lived through, where the frontier agent on memory solves 0.84 and 0.82, at about one fortieth of the list price per solved task; it matches the frontier on success under a change no training trace contains; and a model trained on its own traces improves on its parent. We close with what this implies for how agent platforms should be built, and with the three loops this makes up: outcomes improve the memory, failures become tested and verified repairs to what the agent does, and the validated record becomes the model. That arrangement, in which each loop's output is the next loop's input, is what we mean by recursive self-improvement.

1 Introduction

An agent that starts every task from nothing is expensive. It rediscovers the quirks of its domain on every ticket. It escalates work it could have done with yesterday's knowledge. And in domains

where the textbook is wrong, it applies the textbook. Memory layers exist for the first problem: a vector table, a hosted memory product, or both, so that a note written after one episode can be read in the next.

Our starting point is that recall is necessary and not enough, because the world an agent works in does not hold still. A refund policy is tightened. A carrier changes its API and the old exception handling starts returning the wrong answer. A repository adopts a new test convention. A customer segment that used to respond to a discount stops responding. None of this is announced to the agent, and none of it shows up as a change in the store. A memory layer knows when an entry was written, by whom, and how close it is in embedding space to the current query. It does not know whether the last agent that acted on that entry succeeded, because nothing tells it. When a support playbook says "refund the double charge" and the refund was unwarranted, the ticket reopens and the playbook keeps ranking first. When the operations team switches carriers and yesterday's fix becomes wrong, the store keeps serving yesterday's fix with the same confidence as before.

The risk this creates is not that the agent gets some tasks wrong. Every agent does. It is that the errors are systematic and confident. The same stale rule is applied to every instance of the affected task class, the agent reports each one as handled, and the failures surface later and elsewhere: as reopened tickets, as refunds that should not have been issued, as denials a supervisor has to overturn, as retention offers spent on accounts that were never going to leave, as a rising escalation queue that the humans behind the agent have to absorb. Because the store is what recommended the rule, the agent will keep recommending it until someone finds the entry and deletes it. In our data a memory layer without outcome feedback made this worse, not better: after a rule change the agents with memory applied the stale fix on twice as many first attempts as the agent with no memory, which at least had nothing to be confidently wrong about. Memory degrades quietly in exactly the situations where a production agent most needs to adapt, and the more the team has relied on it, the more it has to lose.

By *learning from outcomes* we mean closing that gap, so that adapting to change is something the system does and not something a human does to it. (Continual learning, in the machine-learning sense of adapting to a task stream whose rules change, is the nearest term of art; we use it sparingly because nothing in this loop updates a weight.) The outcomes of the agent's work, the reopened ticket, the reviewer's rejection, the incident whose symptoms persist, are credited back to the specific memories the agent read and the specific actions it took. Evidence accumulates per entry and per action, and it changes what the next agent is shown. A rule that has kept working is presented as validated. A rule that used to work and has started failing is flagged as a regime shift. A rule that failed repeatedly is shown as a warning, not as a candidate. And the next agent is told which actions were tried on tasks like this one and how each went. No human edits the store for any of this to happen. The layer is the engine by which a fleet of agents adapts to a change none of them was told about.

That loop is the first of three, and it produces the asset the other two run on. Every task leaves a sealed decision trace: the task as posed, the memories read with their versions and evidence status, the actions taken with their results, the attempts abandoned, and the verdict. A slow loop trains a model on the validated part of that record, so that a decision the fleet has learned to make well can be made at a fraction of the price with the memory still on. A repair loop turns a failure in that record into a tested, shipped and verified change to what the agent does. By *recursive self-improvement* we mean this arrangement and nothing grander: the outcomes of an agent's work change what the next agent is told, what it does and, in time, what answers, with no human editing any of it, and each loop's output is the next loop's input, so the system that fixes the agent produces the data that trains the agent. The loops move memory, procedures and

a fine-tuned model. They do not touch a model's architecture or training objective, every change they make is gated on evidence, and every change is reversible.

This paper asks whether closing the first loop changes outcomes for agents as they are actually deployed, with many instances, one shared store, tasks that do not repeat, and rules that change under them, and what it costs; and then what a model trained on the record that loop builds inherits. It is written for people who build agents and the platforms they run on. Section 2 defines learning from outcomes as a contract between an agent and its memory layer and says how it differs from memory. Section 3 describes the benchmark. Sections 4 to 7 give the results in the order of the argument: what no memory costs, what memory buys and where it stops, what learning from outcomes changes for one agent, and what it changes for a fleet. Section 9 reports what a model trained on the outcome record inherits, from our companion study. Section 10 puts the three loops together, and Section 11 gives our recommendations for platform builders.

2 What learning from outcomes is, and how it differs from memory

A memory layer has two operations: write a note, and search for notes similar to a query. Products refine both. Write-time reconciliation lets a new fact update or supersede an old one; there are recency tie-breaks, summarisation, per-user scoping, and sometimes a feedback endpoint. The contract does not change: results are ranked by similarity and freshness, and the store's opinion of an entry changes only when something rewrites it.

A layer that learns from outcomes adds a third operation, commit outcome, and the first two behave differently because of it. In the layer we study the agent contract has four parts.

1. At the start of a task the agent receives a briefing: a compiled digest for the entity it is about to work on, with three evidence sections. *Validated* lists entries that every outcome has confirmed. *Discredited* lists entries a failure has gated, with what replaced them where that is known. *Regime shift* lists long-validated entries that have recently started failing. Every row carries an evidence status: untested, validated, contested or discredited.
2. Retrieval ranks on similarity and on the entry's outcome record for tasks like this one. Discredited entries are removed from the candidates and returned as an explicit avoid list. When the task ends in a choice among a fixed set of actions, the response includes action priors (how each action has fared here on similar tasks) and a recommendation: *act* on an action with a good record, *explore* an untried one when what has been tried is failing, *escalate* when every candidate has been tried and failed.
3. Every domain tool call is recorded under a stable action key. When a remembered approach does not work and the agent moves on to another, it marks the attempt as failed, so the memories behind that attempt receive the failure and only what the agent reads afterwards is credited with the eventual success.
4. At the end of the task the environment's verdict is committed and credited to the memories the agent cited per attempt and to the actions it took. Confidence moves on evidence (a Beta posterior over successes and failures, scaled by surprise, with first-strike tolerance for rules with a long validated history), and the same evidence drives the briefing sections, the ranking and the priors of the next task.

The difference between the two kinds of layer is not the presence of a feedback endpoint. Mem0 has one and we call it on every retrieved memory at outcome time. The difference is what the

layer does with the signal: whether it reaches the specific memories and actions in the causal chain, whether it changes what the next agent is shown and told, and whether the layer can tell a rule that stopped working from a rule that never worked.

Because SenseLab is our product, every comparison in this paper includes a control on the same platform: `senselab-nofeedback`, with identical briefing, identical retrieval and identical reflection notes, and commit outcome never called. It is a strong memory layer with the loop cut. Where it matches the full arm, the loop is not doing the work. Where it does not, the gap is what learning from outcomes adds on top of memory.

3 The benchmark

Arms. Every arm runs the same agent: the same model, system prompt and tool schema (`memory_search`, `memory_write`, the scenario's domain tools, `finish`/`escalate`), the same seeds and task order, a fresh store per cell, and the same post-episode reflection turn in which the agent writes one note for its future self. An episode allows up to three attempts; a task not solved in three is escalated to a human. The only thing that differs is what sits behind `memory_search` and what happens to the store when the episode ends.

`none` no memory. The floor: what the model knows, plus what the environment's feedback teaches it within one episode.

`mem0` Mem0's hosted memory (Pro plan) through its documented API, with write-time reconciliation and its `feedback` API called at outcome time on every retrieved memory, so the product receives every outcome signal it exposes. Ingestion is asynchronous; the harness waits for writes to land before the next episode and reports the wait separately.

`senselab-nofeedback` SenseLab with the outcome loop off, as described above.

`senselab` SenseLab, production API, wired per the contract in Section 2.

Scenarios. Ten scenarios model what teams deploy agents for, weighted by adoption: customer service, software engineering, operations, sales and retention, incident response, personal assistance, analytics, policy Q&A, account management and multi-agent routing. All follow the same rules. No episode repeats; every task is a new instance of a class (a new ticket, a new failing PR, a new order exception). The domain has hidden quirks where the generic answer is wrong: a "double charge" at Acme is a pending authorisation to explain, not to refund; integration-test failures are rerun, not fixed; the specialist whose name says "billing" lost the credentials for it last quarter. Offline checks confirm that an agent that always applies the generic playbook succeeds on 0 to 45% of episodes, so there is room to learn and nothing to look up. Failure feedback is what the real system would say (a reopened ticket, a reviewer rejection, "symptoms persist"), never the answer key. And the store starts populated with 300 realistic but irrelevant operational entries, so retrieval has to discriminate against a production-sized store. This paper reports the six scenarios on which every arm has completed its cells.

support tier-1 resolution, eight issue types; in six the Acme fix differs from the playbook. An unwarranted refund is a critical failure.

ci-fix a coding agent keeping CI green under hidden repo conventions; wrong shortcuts go green and are rejected in review.

order-ops e-commerce exceptions under hidden fulfilment and carrier rules; a wrong denial is overturned by a supervisor and counts as critical.

retention one retention action per hidden customer segment; stable accounts need nothing and any offer is wasted spend.

concierge a personal assistant choosing among options for one executive under hidden preferences (budget, dietary, timing) that drift.

analytics text-to-SQL reporting against a warehouse with quirks: amounts in cents, a misspelt status, test accounts and soft-deleted rows to exclude, refunds in another table.

The other four (*diagnose*, *dispatch*, *accounts*, *policy-qa*) have completed baseline cells and are being completed for SenseLab; they will be added to this report when done.

Regime change. Each single-agent and fleet cell runs 60 episodes. In the four scenarios with a regime change (support, ci-fix, order-ops, retention), the rules change at episode 20 for half of the task classes: the fix that worked becomes wrong and a new, equally non-obvious fix becomes right. Everything the agent has learned about those classes is now stale and still ranks first. This is what production teams describe when they say memory "went bad": a policy changed, a carrier changed its API, an upstream service was re-platformed. We measure recovery on the changed classes over episodes 20 to 59 and report the unchanged classes and the pre-change learning curve alongside. Concierge changes one preference at episode 12; analytics does not change.

Fleet and transfer protocols. In *fleet-6*, six agent identities take the 60 episodes in turn on one shared store, so each sees only ten episodes directly and has to learn the rest from what its peers left behind. *Fleet-10* is ten agents over 100 episodes with the change at 30. In the *pioneer* protocol one agent works alone for 20 episodes and five peers join at 20 (no regime change); we compare the followers' first exposure to each task class against the pioneer's. In the *newcomer* protocol five agents live through a regime change at 20 and a sixth arrives at 40 with no history and takes every episode from then on; the store it inherits holds the old fix, which its peers learned and then burned on, and whatever replaced it. *Cold start* is the same on a healthy store with no change. *Cross-model onboarding* builds the store with veterans on one model and runs the newcomer on the other.

Protocol and statistics. Two frontier models (`gpt-5.5` and `claude-opus-5`), six seeds per cell, fleet sizes 1 and 6 in the core grid, three attempts per episode, a blind judge model for free-text grading. All memory arms hit the production SenseLab and Mem0 APIs. The unit of independence is a cell, one store for one arm, scenario, model, fleet size and seed; episodes inside a cell are not independent, so every headline interval is a cell-level bootstrap with 2,000 resamples, and every two-arm comparison is paired on (scenario, seed, model, fleet): the same task order under the same seed, so the scenario mix and the seed's luck cancel. Coverage by arm: the no-memory and SenseLab arms ran every protocol. The memory product ran the regime-change core, the pioneer protocol and, of the newcomer protocol's four scenarios, support and ci-fix; its dispatch and policy-qa newcomer cells, and the cold-start and cross-model protocols, were not run in this version, and the corresponding tables say so.

4 No memory

Table 1 is the learning curve before the rules change. The no-memory row is flat: 0.53 first-attempt success in the first five episodes, 0.51 in episodes 15 to 19. The agent sees eight or ten instances of the same task class in twenty episodes, is told each time that its textbook answer did not work, and cannot carry the correction to the next instance. Over the whole horizon (Table 17) it solves 52% of tasks on the first attempt, takes 1.68 attempts per task, and escalates 110 of every 1,000 tasks to a human. In the support scenario after the change it escalates 97% of the affected tasks (Table 7). Nothing it knows applies and it has no way to find out what does.

arm	ep 0-4	5-9	10-14	15-19	escalated
no memory	0.53	0.53	0.53	0.51	0.05
memory (Mem0)	0.54	0.51	0.5	0.51	0.07
SenseLab, outcome loop off	0.59	0.73	0.77	0.81	0.04
SenseLab, outcome loop on	0.58	0.7	0.78	0.84	0.04

Table 1: Before the change: first-attempt success on all tasks by 5-episode block, episodes 0–19.

5 Memory: what it buys and where it stops

Two things in Table 1 deserve attention. The SenseLab platform with its loop off learns: 0.59 to 0.81 over twenty episodes, the same curve as the full arm (0.58 to 0.84). That is memory doing its job. The agent's own reflection notes, compiled into a briefing and retrieved with the right entity scope, carry the Acme quirks from one instance of a class to the next. Mem0, on this harness, does not learn: 0.54 to 0.51. Its write-time reconciliation and asynchronous ingestion were built for a different workload (facts about a user, accumulated across conversations) than this one (operational lessons about a domain, needed on the very next task). The arm receives the same reflection notes and the same feedback calls, and what comes back does not move first-attempt success. We report this as measured. It says something about the product's fit to this workload and nothing about memory in general; the no-feedback SenseLab arm shows what a memory layer fitted to the workload achieves.

After the change, memory stops helping. Table 2 is the headline condition: the changed task classes, episodes 20 to 59, single agent. Mem0 and no memory both sit at 0.38 first-attempt success. The SenseLab arm with the loop off does a little better at 0.41, and its stale-pick rate is the highest of the four at 0.18: it keeps recommending the fix that used to work, because nothing in the store can learn that it stopped working. Its recovery curve (Table 3) rises from 0.34 to 0.47 over forty episodes, and that rise comes from the agent's reflection notes, in which the agent writes down that the old fix failed. The store's own opinion of the old fix never changes.

This is where memory stops. A memory layer can hold the old fix and the new one side by side. It cannot rank them by which one is currently true, because "currently true" is an outcome, and outcomes are outside its model of the world. Production teams know the symptom: memory goes bad after a policy change and someone has to go and clean it up.

What it looks like when the agent cannot adapt. The stale-pick column in Table 2 is the one to read for risk. After the change, the agent with no memory applies the pre-change fix on

7% of first attempts; it has no store to tell it what worked before, so its errors are spread across whatever the model guesses. The memory arms apply the stale fix on 15 to 18% of first attempts, twice as often, and the arm with the strongest memory has the highest rate. Those are not random mistakes. They are the store doing its job on a rule that is no longer true: retrieving it, ranking it first, and presenting it with the same confidence it had when it was right. In the support scenario the consequence is visible in the escalation column of Table 7: after the change, 97% of the affected tickets under no memory and 71% under Mem0 end up with a human, because three attempts are not enough to unlearn a playbook the store keeps recommending. Every one of those is a ticket the agent was deployed to handle. In a deployment the error would also not be confined to one agent. Every instance reading the same store inherits the same stale rule and applies it to its own share of the traffic until the entry is found and removed, and the store has no way of knowing which of its thousands of entries that is.

arm	n	pre 1st	post-change 1st-attempt [95% CI]	success	stale-pick	escalated	attempts
no memory	952	0.53	0.38 [0.30,0.45]	0.63	0.07	0.37	2.1
memory (Mem0)	955	0.51	0.38 [0.32,0.44]	0.66	0.15	0.34	2.09
SenseLab, outcome loop off	840	0.72	0.41 [0.35,0.47]	0.68	0.18	0.32	1.99
SenseLab, outcome loop on	825	0.72	0.54 [0.50,0.58]	0.77	0.15	0.23	1.77

Table 2: After the world changes: changed-class tasks, episodes 20–59, single agent. First-attempt success carries a cell-level bootstrap 95% interval; *stale-pick* is the share of first attempts that applied the pre-change fix.

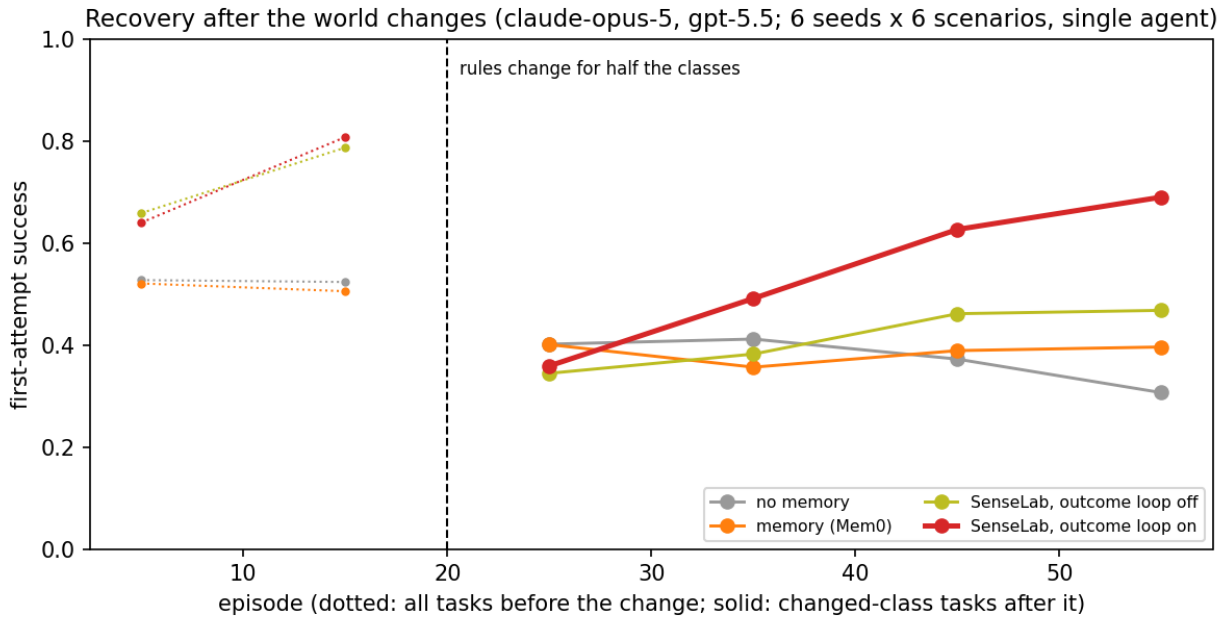


Figure 1: Before the change, memory and outcome learning learn at the same rate and no memory does not (dotted: all tasks). After the change, only the arm with the outcome loop recovers (solid: changed-class tasks, first-attempt success by 10-episode block, single agent, both models, six scenarios).

6 Learning from outcomes: a single agent

This is the adaptation the previous section said memory cannot do on its own. The full arm shares the loop-off arm's pre-change curve and separates from it after the change. On the changed classes it goes from 0.36 in episodes 20 to 29 to 0.49, 0.63 and 0.69 in the following blocks (Table 3). Its stale-pick rate falls from 0.22 to 0.10 (Table 4) while the loop-off arm's stays between 0.14 and 0.21 and Mem0's at 0.15. Thirty-eight of its 42 cells recover, meaning three consecutive first-attempt successes on a changed class, against 25 of 42 with the loop off and 26 of 48 for Mem0. It escalates 23% of the affected tasks; the others escalate 32 to 37%.

Paired on the same cells (Table 5), the arm with the loop on is ahead of Mem0 by 0.16 first-attempt success over episodes 20 to 59 and by 0.28 over episodes 40 to 59, with a 95% interval of $[0.20, 0.37]$. It is ahead of no memory by 0.17 and 0.33, and ahead of its own loop-off control by 0.12 and 0.18, interval $[0.10, 0.25]$, ahead in 18 of 24 paired cells. That last figure is the paper's central result. Same platform, same retrieval, same notes: the outcome loop on its own is worth 18 points of first-attempt success in the last twenty episodes after a regime change, and the interval does not include zero.

arm	ep 20-29	30-39	40-49	50-59	cells recovered (3 in a row)	median-ish exposures
no memory	0.4	0.41	0.37	0.31	24/48	4.1
memory (Mem0)	0.4	0.36	0.39	0.4	26/48	5.3
SenseLab, outcome loop off	0.34	0.38	0.46	0.47	25/42	8.0
SenseLab, outcome loop on	0.36	0.49	0.63	0.69	38/42	8.5

Table 3: Recovery after the change: first-attempt success on changed classes by 10-episode block; cells recovered = three consecutive first-attempt successes on a changed class.

arm	ep 20-29	30-39	40-49	50-59
no memory	0.07	0.08	0.08	0.05
memory (Mem0)	0.14	0.15	0.15	0.15
SenseLab, outcome loop off	0.17	0.21	0.19	0.14
SenseLab, outcome loop on	0.22	0.17	0.1	0.11

Table 4: Stale-pick rate after the change: share of first attempts applying the pre-change fix, by block.

comparison	ep 20-59 delta [95% CI]	cells with senselab ahead	ep 40-59 delta [95% CI]
vs no memory	+0.165 [+0.08,+0.24]	17/24	+0.331 [+0.22,+0.43]
vs memory (Mem0)	+0.158 [+0.09,+0.23]	19/24	+0.283 [+0.20,+0.37]
vs SenseLab, outcome loop off	+0.119 [+0.07,+0.17]	18/24	+0.181 [+0.10,+0.25]

Table 5: SenseLab (outcome loop on) minus each arm, paired on (scenario, seed, model, fleet): post-change changed-class first-attempt success, episodes 20–59 and 40–59.

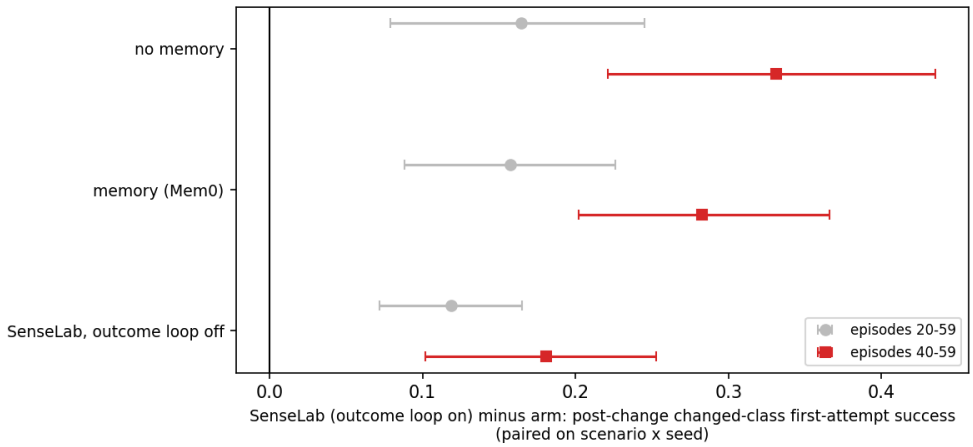


Figure 2: SenseLab with the outcome loop on, minus each arm, paired on (scenario, seed, model, fleet), episodes 40–59; bars are cell-level bootstrap 95% intervals.

By scenario. Table 6 shows where the gain comes from. Support is the clearest case, at 0.45 against 0.08 for Mem0 and 0.21 with the loop off: its changed classes are choices from a fixed set of actions, which is what action priors and the recommendation express. Ci-fix follows (0.62 against 0.47 to 0.50), and its escalation rate falls to 29% where every other arm sits at 45%. Retention (0.54 against 0.44 to 0.48) and order-ops are within noise of the memory arms: their correct action is a conditional rule over several account or order attributes, and per-action evidence gives a marginal over those attributes rather than the rule itself. Section 10 says what we are doing about that.

arm	support	ci-fix	retention	order-ops
no memory	0.0	0.55	0.33	0.63
memory (Mem0)	0.08	0.47	0.44	0.54
SenseLab, outcome loop off	0.21	0.5	0.48	0.51
SenseLab, outcome loop on	0.45	0.62	0.54	0.52

Table 6: Post-change changed-class first-attempt success by scenario, single agent.

arm	support	ci-fix	retention	order-ops
no memory	0.97	0.45	0.03	0.0
memory (Mem0)	0.71	0.45	0.12	0.06
SenseLab, outcome loop off	0.55	0.45	0.11	0.01
SenseLab, outcome loop on	0.41	0.29	0.09	0.01

Table 7: Post-change changed-class escalation rate by scenario, single agent.

What the loop did. In the main single-agent run the SenseLab stores ended with about 19,700 entry versions over 3,300 keys. Some 7,100 versions carried a validated status, 1,900 contested and 2,900 discredited; 1,100 keys ended discredited. Those are the pre-change fixes the change made wrong, and the agent's wrong guesses on the way to the new ones. Each was removed from the candidate list and shown as a warning by the evidence, with no human involved, and each validated row is one the next agent could act on without re-verifying. Table 8 shows the behavioural side: on each changed class the agents with the loop on tried more distinct actions (6.3 per class against

4.7 to 5.8), in fewer attempts (16.4 against 19.4 to 20.6), and reached the new fix on a larger share of classes (299 of 353, against 238 of 353 with the loop off and 233 of 394 with no memory).

arm	(cell, class) pairs	attempts / pair	distinct actions / pair	stale final picks / pair	pairs that ever tried the new fix	pairs solved at least once
no memory	394	20.6	4.7	0.4	233/394	235/394
memory (Mem0)	395	19.9	5.6	1.3	270/395	273/395
SenseLab, outcome loop off	353	19.4	5.8	1.6	238/353	250/353
SenseLab, outcome loop on	353	16.4	6.3	0.8	299/353	306/353

Table 8: What the fleets tried on each changed class after the change (all fleet sizes).

7 Learning from outcomes: a fleet

One agent that learns is useful. A platform runs many, and the question for a platform builder is whether what one agent learns reaches the others without anyone copying it across.

Six agents, one store. In the fleet-6 cells each agent sees ten of the sixty episodes and at most one or two instances of any changed class. Table 9 shows the fleet matching the single-agent result: 0.57 first-attempt success on changed classes after the change against 0.54 for one agent, 20% escalated against 23%. Mem0's fleet is at 0.36 (single agent 0.38) and no memory's at 0.37. Knowledge that lives in outcomes travels because outcomes are shared. When agent 3 burns on the old fix, agent 5's briefing carries the discredit, and agent 5's success validates the replacement for agent 1.

Ten agents, one store. Table 10 scales this to ten agents over a hundred episodes, with the change at 30, on four scenarios (support, ci-fix, policy-qa and dispatch, the last two harder than the core six). Each agent sees ten episodes, so most agents meet a changed class once or twice at most. The fleet with the loop on solves 0.59 of changed tasks on the first attempt after the change and escalates 29%; the same platform with the loop off is at 0.37 and 43%, Mem0 at 0.30 and 54%, no memory at 0.22 and 62%. The stale-pick rate tells the same story from the other side: 0.04 for the learning fleet, 0.14 for both memory arms without the loop. The column that matters most for a platform is each agent's first exposure to a changed class after the change, the moment it has to rely on what others learned: 0.39 with the loop on, 0.28 with it off, 0.21 on Mem0, 0.23 with nothing. Ten writers on one store made the loop's job harder, since more of them burn on the old fix before it is marked, and the result is the same as with six.

arm	n	single-agent 1st	fleet 1st	fleet stale	fleet escalated	each agent's FIRST post-change exposure: 1st-attempt	...stale
no memory	951	0.38	0.37	0.07	0.36	0.39	0.05
memory (Mem0)	954	0.38	0.36	0.16	0.35	0.36	0.15
SenseLab, outcome loop off	828	0.41	0.42	0.2	0.32	0.38	0.21
SenseLab, outcome loop on	840	0.54	0.57	0.12	0.2	0.42	0.19

Table 9: Six agents sharing one store: post-change changed-class tasks. Each agent sees ten of the sixty episodes; *first post-change exposure* is an agent's first task on a class after the change.

arm	n	single-agent 1st	fleet 1st	fleet stale	fleet escalated	each agent's FIRST post-change exposure: 1st-attempt	...stale
no memory	1637	0.28	0.22	0.01	0.62	0.23	0.01
memory (Mem0)	1669	0.28	0.3	0.14	0.54	0.21	0.16
SenseLab, outcome loop off	1680	0.35	0.37	0.14	0.43	0.28	0.16
SenseLab, outcome loop on	1680	0.54	0.59	0.04	0.29	0.39	0.08

Table 10: Ten agents sharing one store, 100 episodes with the change at 30: post-change changed-class tasks.

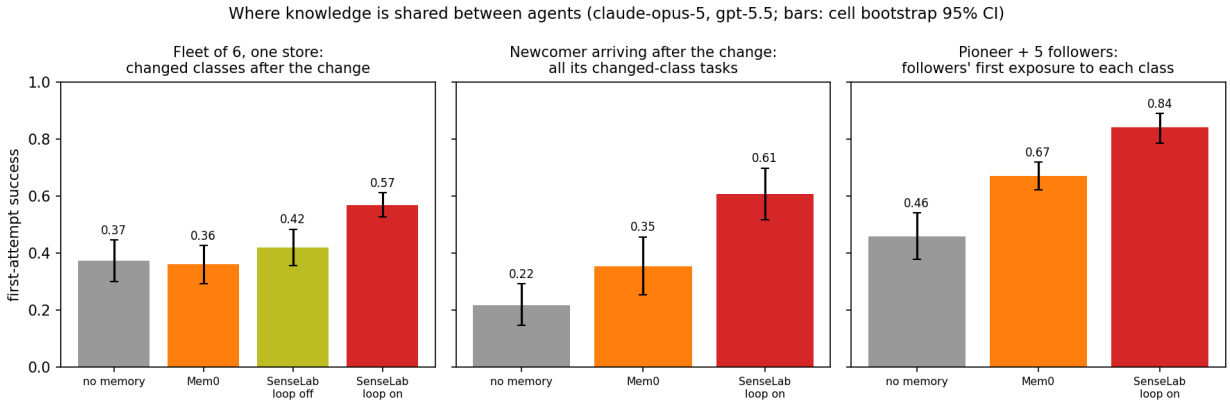


Figure 3: Where knowledge is shared between agents: a fleet of six on one store after the change; a newcomer's changed-class tasks after arriving post-change; followers' first exposure to each class after a pioneer learned alone.

A newcomer after the change. The transfer protocols isolate the adoption question. In the newcomer protocol five agents live through the regime change; a sixth arrives twenty episodes later with no history and inherits a store in which the old fix was learned, burned on and replaced. Table 11 reports the newcomer's first exposure to each changed class, the first time it has ever seen the task. The three-arm comparison is on the two scenarios every arm completed, support and ci-fix, with the same 24 cells, seeds and models per arm. There the no-memory newcomer solves 0.20 of its first exposures and escalates 77%. The Mem0 newcomer solves 0.22 $[0.13, 0.32]$ and escalates 53%, with the highest stale-pick rate of the three at 0.15: it inherits a store that still ranks the old fix first, and it applies it. The newcomer on the SenseLab store solves 0.62 $[0.52, 0.72]$ and escalates 32%; the store it inherited had already discredited the old fix. The rows marked *all* give the two arms that ran all four scenarios: 0.49 $[0.40, 0.59]$ against 0.19 over 140 first exposures, escalations 39% against 65%, stale-pick 0.05, and 79% of the keys the SenseLab newcomer cites written by its peers. Dispatch and policy-qa are the harder half for the newcomer (0.40 against 0.19 with no memory), so the matched subset is the more favourable one and the four-scenario rows are the ones to quote for the platform alone; Mem0's dispatch and policy-qa cells are reported in the next version. Table 12 shows what happens afterwards on all four scenarios. Over its twenty episodes the newcomer goes from 0.55 to 0.68 on changed classes while the no-memory newcomer stays near 0.2; by its last block it is ahead of where the veterans were before it arrived (0.49 in episodes 30 to 39). It did not live through the change and did not have to.

arm	scenarios	cells	1st-exposures (changed)	1st-attempt [95% CI]	stale-pick	escalated
no memory	ci-fix + support	24	60	0.2 [0.11,0.31]	0.0	0.77
memory (Mem0)	ci-fix + support	24	60	0.22 [0.13,0.32]	0.15	0.53
SenseLab, outcome loop on	ci-fix + support	24	60	0.62 [0.52,0.72]	0.1	0.32
no memory	all (4)	48	140	0.19 [0.13,0.27]	0.02	0.65
SenseLab, outcome loop on	all (4)	48	140	0.49 [0.40,0.59]	0.05	0.39

Table 11: Newcomer after a regime change: five agents live through the change at 20; a sixth arrives at 40 with no history and takes every episode from then on. The first three rows cover the scenarios every arm completed (support and ci-fix; same cells, seeds and models); the rows marked *all* cover the four scenarios of the protocol for the arms that ran all of them.

arm	1st 40-44	45-49	50-54	55-59	stale 40-44	45-49	50-54	55-59
no memory	0.2	0.23	0.25	0.19	0.02	0.01	0.02	0.0
SenseLab, outcome loop on	0.55	0.55	0.66	0.68	0.04	0.02	0.06	0.05

Table 12: The newcomer's first-attempt success on changed classes by 5-episode block after arrival, all four scenarios (arms that ran all of them).

Followers joining a pioneer. Table 13 is the mirror image: one agent discovers the quirks alone for twenty episodes, five followers join, and the followers' first exposures show what transferred. The pioneer itself does about equally well on every arm (0.45 to 0.59 on its first exposures; it is discovering, and the store cannot help much with that). The followers are where the arms separate. With no memory they solve 0.46 of their first exposures, the same as the pioneer did, because nothing transferred. On Mem0 they solve 0.67 $[0.62, 0.72]$. On the SenseLab store they solve 0.84 $[0.78, 0.89]$ and escalate 10% against 13% and 25%; 83% of the keys they cite were written by the pioneer or by each other. Table 14 shows that this is not a slow ramp: the followers are at 0.85 in their first ten episodes and stay there, while the Mem0 followers climb from 0.63 to 0.74 over thirty episodes as they re-learn part of what the pioneer already knew. The difference between the two memory layers here is the evidence attached to the pioneer's notes. The follower on the SenseLab store is reading notes that twenty outcomes have confirmed or discredited, and it treats them accordingly.

arm	pioneer 1st-exposures	1st-attempt	escalated	follower 1st-exposures	1st-attempt [95% CI]	escalated	transfer ratio	peer-authored share of cited keys
no memory	456	0.45	0.3	962	0.46 [0.38,0.54]	0.25	1.02	-
memory (Mem0)	455	0.55	0.17	936	0.67 [0.62,0.72]	0.13	1.22	0.81
SenseLab, outcome loop on	456	0.59	0.13	962	0.84 [0.78,0.89]	0.1	1.42	0.83

Table 13: Pioneer protocol: one agent alone for 20 episodes, five peers join at 20. First exposure = an agent's first episode on a task class it has never handled.

arm	ep 20-29	ep 30-39	ep 40-49
no memory	0.46	0.44	0.46
memory (Mem0)	0.63	0.66	0.74
SenseLab, outcome loop on	0.85	0.84	0.85

Table 14: Followers after joining: first-attempt success on all their episodes by 10-episode block.

Cold start onto a healthy store. Table 15 removes the regime change: five agents work for forty episodes on a store nothing has gone wrong in, and a sixth arrives. The no-memory newcomer solves 0.47 of its first exposures. The newcomer on the SenseLab store solves 0.78 $[0.71, 0.84]$, level with the veterans' own rate in their last ten episodes before it arrived (0.76), and 82% of the keys it cites are its peers'. Escalations are close for both newcomers (6% and 8%): on a healthy store the tasks have answers, and the no-memory agent finds them eventually too. The difference is in getting them right the first time. This is the adoption case in its simplest form: a new agent, or a new instance, is productive on its first task because the store it joins carries the fleet's validated knowledge, and there is nothing for it to discover on its own. Mem0 was not run for this protocol in this version; the table compares the platform with no memory only.

arm	veterans ep30-39 1st	1st-attempt [95% CI]	escalated	newcomer all ep40+ 1st
no memory	0.42	0.47 [0.43,0.51]	0.08	0.44
SenseLab, outcome loop on	0.76	0.78 [0.71,0.84]	0.06	0.78

Table 15: Cold start onto a healthy store (no regime change): five agents for 40 episodes, a sixth arrives at 40.

Crossing models. Table 16 repeats the cold start with the store built by veterans on one model and the newcomer on the other. Changing the newcomer's model costs nothing measurable: a `gpt-5.5` newcomer on a store built by `claude-opus-5` veterans solves 0.73 of first exposures against 0.74 for a same-model newcomer, and a `claude-opus-5` newcomer on a `gpt-5.5` store solves 0.82 against 0.82. What does move the number is which model built the store: both newcomers land at 0.82 on the `gpt-5.5` store and at 0.73–0.74 on the `claude-opus-5` store, whichever model they run themselves. Without memory the four combinations sit at 0.46–0.47. Escalations stay between 5% and 10% in every SenseLab cell and 11 or 12 of 12 cells reach parity within five episodes in every row. The knowledge in the store is written in plain language, with its evidence attached, and a different model reads it as well as the one that wrote it. For a platform this means the fleet's store outlives any single model choice: routing a task to a cheaper or newer model does not reset what the fleet has learned. Mem0 was not run for this protocol in this version.

arm	veterans' model	newcomer model	1st-attempt	escalated	reached parity	never (share)
no memory	claude-opus-5	claude-opus-5 (same)	0.47	0.09	11/12	8% (n/a within 20)
no memory	claude-opus-5	gpt-5.5	0.47	0.06	11/12	8% (n/a within 20)
no memory	gpt-5.5	claude-opus-5	0.47	0.1	11/12	8% (n/a within 20)
no memory	gpt-5.5	gpt-5.5 (same)	0.46	0.08	11/12	8% (n/a within 20)
SenseLab, outcome loop on	claude-opus-5	claude-opus-5 (same)	0.74	0.05	12/12	0% (n/a within 20)
SenseLab, outcome loop on	claude-opus-5	gpt-5.5	0.73	0.05	11/12	8% (n/a within 20)
SenseLab, outcome loop on	gpt-5.5	claude-opus-5	0.82	0.1	11/12	8% (n/a within 20)
SenseLab, outcome loop on	gpt-5.5	gpt-5.5 (same)	0.82	0.05	12/12	0% (n/a within 20)

Table 16: Cross-model onboarding onto a healthy store: five veterans on one model for 40 episodes, a newcomer on the other model arrives at 40. Same-model rows of the same arm are the reference.

8 What it costs

Learning from outcomes reads more on every task than no memory does: a briefing, retrieved context, and the evidence rows that come with them. Over the whole horizon (Table 17) what that buys per 1,000 tasks is 240 more first-attempt solutions than no memory and 180 more than Mem0, 40 fewer escalations to a human than either Mem0 or no memory (20 fewer than the same platform with the loop off), and a third of an attempt saved per task against no memory. Whether the extra reading is worth 40 avoided escalations per 1,000 tasks depends on what an escalation costs the team that deploys the agent; for support, operations and engineering workflows a human touch is rarely the cheap side of that comparison.

arm	n	1st-attempt [95% CI]	eventual success	escalations / 1000	attempts / task	wall s / task	memory s / task
no memory	4286	0.52 [0.48,0.56]	0.89	110	1.68	6.2	0.0
memory (Mem0)	4307	0.58 [0.55,0.62]	0.89	110	1.62	30.9	3.8
SenseLab, outcome loop off	3894	0.75 [0.71,0.79]	0.91	90	1.38	22.7	11.4
SenseLab, outcome loop on	3939	0.76 [0.72,0.80]	0.93	70	1.35	32.7	22.3

Table 17: Whole horizon, every task (episodes 0–59, single agent): outcomes per 1,000 tasks. Memory time is the time spent in memory calls; SenseLab timings are those of the production API under the grid's full load.

The SenseLab wall-clock and memory-time figures are those of a production API under the full load of this grid, up to a hundred cells in flight alongside other workloads, network round trips included. They are an upper bound, not the product's latency at rest. Outcomes are unaffected.

9 A model trained on what the agents learned

The loop above changes what the next agent reads. It also leaves, as a side effect of working, one sealed decision trace per task, and a trace that ends in a verdict is a supervised example. Our companion study [3] asks what happens when those traces train a model. We summarise its results here because they are the second half of the argument: the record the outcome loop builds is how a fleet adapts, and it is also the training set of the model that eventually makes the fleet's frequent decisions. The numbers below are the companion study's; the setup in full, the ablations and every table are there.

Setup. The two frontier agents of this paper worked two of its scenarios, support and ci-fix, for 60 episodes each on the SenseLab arm, through the same regime change at episode 20. Each teacher's sealed traces (about 720 per scenario and model, 81 to 87% of them successes, the rest failures with the failing attempt marked) trained a SenseLab Managed Model: a small model tuned on those traces and on nothing else in the account, scoped by the memory paths the teacher's cells used. A training row is what the teacher saw when it decided: the task, the memory entries retrieved with their evidence status, the actions already tried in the task and how they fared, and the outcome record's recommendation; the target is the teacher's last consequential action. The student is served through an inference gateway that retrieves from the evaluation cell's own store, renders the same prompt shape, and seals each decision as a trace. It never runs the tool loop: one action per call, executed by the environment, with a failed attempt returned to it as a failed tool result. A version is promoted only if it clears the product's held-out bars on agreement with the teacher's decisions, parse rate and failure reproduction. Ten students were trained, one per scenario and teacher in two generations, and evaluated on held-out seeds against the frontier teacher under the same harness and at list prices, with fallback to the frontier switched off so that a student's failures are its own.

What the student inherits. Under the regime change the teacher lived through, the student with memory solves 0.94 and 0.92 of tasks on the two scenarios where the frontier agent on the same memory solves 0.84 and 0.82, at $0.0014 against $0.061 per solved task, about one fortieth, and about 19 seconds an episode faster. It is ahead of its teacher here for a reason the companion study is careful about: the teachers' traces from episodes 20 to 60 show the new fix winning, so both regimes are in the weights and the memory's job is to say which one the current task is in. That is a weak test of memory at inference, so the study adds a second change set that no training trace contains and runs the same students under it with nothing retrained. There the student matches the frontier on success within noise, at $0.0010 to $0.0022 against $0.044 to $0.047 per solved task,

and recovers about seven tenths of the frontier's first-try gain from memory. The frontier is still the better agent on that harder change, which is one reason it stays in the loop.

What the memory still does. The memory keeps working after the experience is in the weights. Switching it off costs the student 0.26 first-try success after the change, under both change sets. Serving the retrieved context without the outcome record (the same entries, without their evidence status, the actions tried, or the recommendation) costs 0.24: most of what the memory does for a tuned model is the evidence, and the text carries little of it. A text skill file maintained the way WikiSkill [4] describes, rebuilt behind a validation gate, sits 0.10 below the memory layer after the change and costs more per solved task than the frontier on memory.

A student trained on its own traces. A student's evaluation cells are themselves sealed traces. A third generation trained on the second generation's own traces, with no frontier model anywhere in the loop, improves on its parent: without memory it is 0.10 and 0.19 first-try success ahead on the two scenarios, and 0.20 and 0.42 ahead after the change, because the corrections the parent made with memory in front of it are what the child does first. With memory served the two are within 0.03. The companion study reads this as memory turning into weights one generation at a time. It is a same-change result and the child was not run under the novel change set, so it says nothing yet about a change the second generation has never seen. The same recipe run on the skill-file agent's traces gave a student the gate would not serve on one scenario: its context carried no outcome record, so it could not tell a failed decision from a successful one, and on held-out decisions that had gone wrong it repeated the failed action more often than the gate allows.

Order of operations. Nothing here starts with a small model. The frontier agent on the memory layer is where an account starts, because it is what produces every trace, and on these two scenarios it is worth 0.12 success and 0.19 first-try success over the same agent with no memory. Distillation comes once a decision has volume. The frontier does not leave the loop: it handles every task outside a student's scope, it is the teacher for the next scenario and the next regime, and its traces keep the store learning.

10 The three loops of recursive self-improvement

The loop measured in Sections 6 and 7 is a fast loop. An outcome moves the posterior on the entries and actions in its causal chain, and the next agent reads the result. On failure it subtracts: a bad entry loses confidence, a bad action loses its recommendation, and the agent is told what not to do. What to do instead it still has to find by exploring, and the fleet then learns from the record of that exploration. The recovery curves above show what that looks like. It is real, and slower than it needs to be.

The fast loop produces one asset nothing else in the stack has: a replay buffer of production behaviour with rewards. Every trace holds the task as posed, the memories read with their versions and evidence status, the actions taken with their results, the attempts abandoned, and the verdict. Two more loops close over that record.

The slow loop turns proven trajectories into the model. Section 9 is that loop measured: a student learns the frontier agent's decisions from its traces, keeps the memory on for what the weights do not know, and its own traces train its successor. What the fast loop adds is selection. Only what the outcome record validated is worth learning from, and the record says which that is.

The repair loop turns a failure into a tested repair. Where the fast loop demotes, the repair loop proposes. From a failed trace, or from a group of failures that share a cause, it localises the entry or action at fault and proposes a scoped replacement: a corrected entry, or a procedure with its conditions, steps, verification and things to avoid. The proposal comes with a falsifiable prediction of which past failures it would have fixed and which past successes it must not break. The prediction is replayed against the recorded history under the candidate memory on a branch; the change ships as a reviewable, revertible commit only if it repairs without regressing; production outcomes then verify the prediction, and the change reverts on its own if it was wrong. Every proven repair is a labelled example (situation, wrong action, right action, outcome) for the slow loop. That is the recursive part. The system that fixes the agent produces the data that trains the agent.

The conditional-rule tasks are the first target. A repair agent reading retention's outcome record does not need a better prior over "offer a discount". It needs to write the rule (which segment gets which action), test it against the forty recorded episodes, and ship it as a procedure the next agent reads first. The record it needs is one the outcome loop already holds complete, unfiltered by what the agent chose to write down. The repair loop is now running on the platform. Our next report runs it on the same ten scenarios under the same protocol, with a fixed token budget, held-out scenario families the loop never sees during development, and a public score visible to the loop alongside a private one that decides, against the arms of this paper. It reports success after the regime change, exposures to recover, regressions introduced by repairs, and cost per repaired failure. The claim under test is that an agent's failures can stop being a maintenance cost and become the input to its own improvement.

11 Recommendations for agent and platform builders

1. Close the loop, or expect memory to go bad. Whatever memory layer you run, wire the environment's verdict back to the specific memories the agent read. Without that, the store ranks by similarity and age, and a policy change turns it into something a human has to clean up. The 18-point gap between the same platform with and without the loop is the size of that liability in our data.

2. Credit outcomes to what was actually used, per attempt. Blaming everything the agent read for a failure punishes the note that would have saved it. Cite keys, mark failed attempts, and let only what was read after the last failed attempt take credit for the recovery.

3. Show evidence where the decision is made. An untested 0.9 and a validated 0.9 are different things to act on, so the agent needs the status as well as the score, and it needs to see what was tried here and how it went. Discredited knowledge should come back as a warning, not as a lower-ranked candidate.

4. Recommend only where there is a choice. Action priors and an act/explore/escalate recommendation belong to terminal tools with a fixed action set. On free-form decisions a recommendation to act reads as "follow the top hit", and a strong model will.

5. Treat a win rate as a current figure, not a lifetime one. Detect the losing streak, flag the regime shift, and let exploration unblock after one failure rather than three.

6. Share outcomes across the fleet, and the notes with them. A fleet learns as one when outcomes are shared; with per-agent memories every quirk is learned once per agent. The fleet-6 and

newcomer results match the single agent's because the evidence is scoped to the entity, not to the agent.

7. Price the extra reading against escalations avoided and attempts saved. A no-memory baseline reads less per task and pays for it in humans.

8. Keep the outcome record complete and queryable. Per-entry and per-action evidence does not learn a conditional rule; a rule inducer or a repair loop will need the record to do it.

9. Treat every sealed trace as a training example, and keep the evidence in it. The model in Section 9 was trained on traces nobody labelled by hand, and a student trained on traces without an outcome record could not tell a failed decision from a successful one. When a decision has volume, distill it; keep the memory on for the student, and keep the frontier in the loop as teacher and fallback.

12 Threats to validity

We built one of the arms. The harness, seeds and per-episode records are public, and the scenarios where the outcome loop makes no difference (retention, order-ops) are reported in the same tables as the ones where it does. Mem0's flat pre-change curve is a finding about its fit to this workload, not about memory in general; the loop-off SenseLab arm is the fairer memory comparison and is reported throughout. Six of ten scenarios are reported; the four not yet complete for SenseLab are closer in structure to retention and order-ops than to support. The environments are synthetic and deterministic, which is what makes tens of thousands of outcomes gradable without a human; the quirks are modelled on real deployments but are not real deployments. SenseLab and Mem0 timings include the network round trip and, for SenseLab, an API under the grid's full load. The managed-model results in Section 9 come from two of the six scenarios, the students answer in decision mode without the teacher's diagnostic tools, and the self-trained generation was measured only under the change its parent trained through. The companion study also reports a shortcut its first student found, in ticket and PR identifiers that were counted from the episode index and so encoded the regime, together with the fix; every result quoted here is from runs after that fix.

13 Conclusion

Memory gives an agent yesterday's notes. Learning from outcomes gives it yesterday's results. In production the rules will change, more than once, without notice, and an agent that cannot adapt turns each change into a systematic error applied confidently to every task it touches until a human finds the cause. On tasks that do not repeat, in a world that changes, the difference is between a store that quietly serves a stale fix with full confidence and one that has already marked it discredited, named what replaced it, and told the next agent which actions were tried here and how they went. In our data it is worth 18 points of first-attempt success in the last twenty episodes after a regime change on the same platform with the loop on rather than off, 28 points over a hosted memory product, a quarter to a third fewer escalations to humans, a fleet and a newcomer that inherit the recovery instead of repeating it, and followers who are productive from their first task on a store a pioneer has already tested: 0.84 of their first exposures solved, against 0.67 on the memory product and 0.46 with no memory. The record that loop leaves behind is the rest of the engine. A model trained on it makes the fleet's frequent decisions at a fortieth of the price with the memory still on, and a model trained on that model's traces improves on it. Where the answer

is a rule and not an action, the same record holds what is needed to write the rule down, test it against the past, and ship it, and that is where the repair loop starts. Three loops, each fed by the last one's output: that is what makes the improvement recursive, and the first of them is what this paper measured.

References

[1] `benchmarks/continual_learning/HYPOTHESIS.md`, preregistration, written 2026-09-16 before harness code or data; amendments dated in place.

[2] Harness, scenarios, analysis and per-episode JSONL: `benchmarks/continual_learning/` in the `amfs` repository.

[3] SenseLab Research. *Experience in the weights: distilling a frontier agent's continual-learning memory into a small model, and what the memory still does at inference.* Companion study, September 2026; available on request.

[4] WikiSkill: self-evolving agent skills from experience. Google, 2026.

[5] SenseLab Research. *Memory that survives the router: a head-to-head study of cross-model recall and verifiable provenance for LLM agents behind a model gateway.* July 2026.